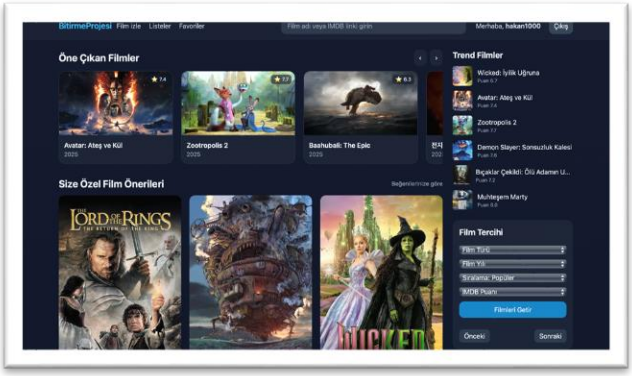
BİTİRME PROJESİ SUNUM KİTAPÇIĞI

Film Öneri ve Keşif Platformu (Bitirme Projesi)

Öğrenci: Hakan Polat - 02225076055

Proje Türü: Web Tabanlı Uygulama + Öneri Sistemi



İçindekiler

1. Proje Özeti
2. Problem Tanımı ve Motivasyon
3. Amaçlar ve Kapsam
4. Kullanıcı Senaryoları
5. Sistem Mimarisi (Genel Tasarım)
6. Kullanılan Teknolojiler ve Nedenleri
7. Veritabanı Tasarımı (PostgreSQL)
8. Öneri Sistemi Nasıl Çalışıyor?
9. Cache ve Performans Yaklaşımı
10. Loglama ve İzlenebilirlik (Observability)
11. Güvenlik Yaklaşımı
12. REST API Kullanımı ve Endpointler
13. Docker ile Kurulum ve Çalıştırma
14. Test Süreci (Selenium + Pytest)
15. CI/CD Durumu ve Önerilen Akış
16. Projenin Güçlü Yanları
17. Limitler ve İyileştirme Fikirleri
18. Sonuç

1) Proje Özeti

Bu proje, kullanıcıların film keşfetmesini ve kişiselleştirilmiş öneriler almasını sağlayan web tabanlı bir platformdur. Kullanıcı; film arayabilir, detay sayfasına gidebilir, favorilerine ekleyebilir, beğeni/puan sinyalleri oluşturabilir ve bu sinyaller üzerinden sistemin ürettiği **kişiselleştirilmiş film önerilerini** görebilir.

Sistem iki temel akış sağlar:

- **Keşif (Discovery):** Tür/yıl/puan filtreleriyle TMDB üzerinden film listeleme.
- **Kişiselleştirme (Personalized Recommendation):** Favori/puan/trailer izleme gibi davranışlardan “kullanıcı profil vektörü” çıkarıp benzer filmleri önerme.

2) Problem Tanımı ve Motivasyon

Günümüzde film platformlarında içerik çok fazla olduğundan kullanıcılar “ne izlesem?” sorusuna hızlı cevap arıyor. Basit listeleme yeterli değil; kullanıcı davranışlarını anlayan ve kişiselleştirme yapan bir sistem daha değerli.

Bu proje ile:

- Kullanıcıların tercihlerinin ölçülmesi,
- Tercihlere göre öneri üretilmesi,
- Performanslı ve gözlemlenebilir (log’lanabilir) bir yapı kurulması hedeflenmiştir.

3) Amaçlar ve Kapsam

3.1 Amaçlar

- Kullanıcı kayıt/giriş sistemi kurmak
- TMDB API ile film verilerini dinamik çekmek
- Favori/puanlama/trailer etkinliklerinden sinyal toplamak
- Embedding tabanlı öneri sistemi kurmak
- Cache + loglama ile performansı ve izlenebilirliği arttırmak
- Uçtan uca testlerle temel UI akışlarını doğrulamak

4) Kullanıcı Senaryoları

Aşağıdaki senaryolar sistemin beklenen davranışlarını tanımlar:

1. **Kayıt Ol / Giriş Yap**
- Kullanıcı e-posta ve şifreyle kayıt olur
- Daha sonra giriş yaparak kişisel alanlarını kullanır
2. **Film Arama**
- Üst arama kutusuna film adı girilir
- Sistem /search sayfasına yönlendirir ve sonuçları listeler
3. **Film Keşfet (Discover)**
4. Tür, yıl, puan filtresi seçilir
5. TMDB /discover endpoint’i üzerinden sonuçlar listelenir

4. **Favori & Puanlama**
- Kullanıcı film detayında “favori”ye ekleyebilir
- Beğendim/Beğenmedim gibi sinyal bırakabilir
5. **Kişiselleştirilmiş Öneri**
- Kullanıcının sinyalleri yeterliyse sistem /api/personalized ile önerileri üretir
- Öneriler belirli süre cache’lenir, sinyal değişirse cache invalid edilir

5) Sistem Mimarisi

Proje temel olarak “modüler monolith” yaklaşımındadır:

- Flask uygulaması içinde blueprint’ler (pages/auth/api) ile katmanlı ayrım
- Services klasöründe iş mantığı (TMDB, recommender, embeddings, events/logging)
- PostgreSQL kalıcı veri saklama
- Docker Compose ile servis orkestrasyonu

Bileşenler:

- **Web (Flask + Gunicorn):** HTTP isteklerini karşılar, template render eder, API döner
- **DB (PostgreSQL):** kullanıcı, favoriler, puanlar, eventler, cache tabloları
- **TMDB API:** film meta datası kaynağı
- **Model/Embedding katmanı:** film vektörleri ve benzerlik hesapları
- **(Opsiyonel) Redis:** dağıtık cache ihtiyacı olursa kullanılmak üzere tasarlandı

6) Kullanılan Teknolojiler ve Nedenleri

6.1 Backend

- **Python + Flask:** hızlı geliştirme, blueprint ile modüler yapı
- **Gunicorn:** production-grade WSGI sunucu, concurrency için gthread worker

- **Requests:** TMDB HTTP istekleri

6.2 Veritabanı

- **PostgreSQL:** ilişkisel tutarlılık, JSON alan desteği, indeksleme ve sorgu gücü

6.3 Öneri Sistemi / NLP

- **Sentence-Transformers / Transformers:** embedding üretimi (film açıklaması/başlık vs.)
- **NumPy:** vektör işlemleri ve cosine/inner product benzerliği

6.4 Frontend

- **Jinja2 template:** backend render
- **TailwindCSS CDN:** hızlı, modern UI
- **Vanilla JS (main.js):** canlı arama önerileri, trailer modal vb.

6.5 DevOps

- **Docker + Docker Compose:** taşınabilir kurulum, tek komutla ayağa kaldırma
- **.env yönetimi:** konfigürasyon ayrımı

6.6 Test

- **Pytest:** test çatı sistemi
- **Selenium:** uçtan uca (E2E) UI doğrulama

7) Veritabanı Tasarımı (PostgreSQL)

Veritabanı tasarımı, kullanıcıların uygulama içindeki etkileşimlerini **"sinyal" (signal)** olarak toplayıp bu sinyallerden **öneri (recommendation)** üretmeye uygun olacak şekilde kurgulanmıştır. Buradaki temel fikir şudur: Kullanıcının **favoriye eklemesi, beğenmesi/beğenmemesi, trailer izlemesi ve yorum yapması** gibi hareketler, kullanıcının film zevkini temsil eden veriler üretir. Sistem bu verileri biriktirerek kullanıcıya daha isabetli öneriler sunar.

7.1 Temel Tablolar ve Rolleri

- **users** Kullanıcı hesabı bilgilerini tutar (username, email, password_hash, created_at). Kimlik doğrulama ve tüm ilişkiler için merkez tablodur.
- **favorites** *(explicit sinyal)* Kullanıcının favoriye eklediği filmleri tutar. UNIQUE(user_id, movie_id) ile aynı film birden fazla kez favoriye eklenemez. Öneri sisteminde güçlü bir "beğeni" sinyalidir.
- **ratings** *(explicit sinyal)* Kullanıcının filme verdiği oyu tutar. value alanı -1 / +1 olacak şekilde kısıtlanmıştır (beğenmedim / beğendim). Bu yapı, klasik puanlamadan daha basit ama karar verdirici bir sinyal üretir.
- **trailer_events** *(implicit sinyal)* Kullanıcının trailer izleme gibi davranışlarını tutar. Bu tablo "kullanıcı açıkça beğendim demese bile ilgilendi" şeklinde **örtük (implicit)** sinyal sağlar. Ayrıca user_id, created_at DESC index'i ile kullanıcı bazlı son event'ler hızlı çekilebilir.
- **comments** *(içerik + duygu analizi)* Kullanıcı yorumlarını tutar ve opsiyonel olarak sentiment_label / sentiment_score alanlarıyla yorumun duygu analizi sonucu saklanabilir. Bu sayede "yorum içeriği" de bir sinyal haline gelebilir.

7.2 Öneri Motorunu Destekleyen Cache/Model Tabloları

- **candidate_movies** *(aday havuzu / cache)* Öneri üretirken değerlendirilecek film havuzunu JSONB olarak tutar. Dış kaynaktan (ör. TMDB) gelen verileri tekrar tekrar çekmek yerine güncel bir cache katmanı görevi görür.
- **movie_embeddings** Film açıklaması/özeti gibi metinlerden üretilen embedding vektörünü saklar (embedding JSONB). text_hash ile metin değişmiş mi kontrol edilir; böylece sadece gerekli olduğunda embedding güncellenir.
- **user_profiles** Kullanıcının sinyallerinden üretilen "profil embedding" bilgisini tutar. signals_hash alanı, kullanıcının favori/rating/trailer sinyalleri değişti mi diye anlamak için kullanılır. Değişmemişse profil yeniden hesaplanmaz (performans kazancı).
- **user_recommendations** *(öneri cache)* Kullanıcıya üretilen öneriler (movie_id, score, data) burada saklanır. signals_hash + updated_at sayesinde "kullanıcı sinyali değişti mi, öneri güncel mi?" kontrolü yapılır. Ayrıca user_id, updated_at DESC index'i ile kullanıcı önerileri hızlı listelenir.

7.3 Loglama / İzlenebilirlik (Opsiyonel ama önemli)

- **user_events** Sistem içinde gerçekleşen olayları (event_type, path, status vb.) ve bazı kimlik bilgilerini **hash'lenmiş** şekilde (ip_hash, ua_hash) kaydeder. Bu tablo; hata analizi, kullanım istatistiği ve güvenlik izleme gibi amaçlara altyapı sağlar.

```
hakan@192 bitirmeProjesi % docker compose exec db psql -U postgres -d film_app

psql (16.11)
Type "help" for help.

film_app=# docker compose exec db psql -U postgres -d film_app
film_app-# \dt
                List of relations
 Schema |         Name          | Type  |  Owner
--------+-----------------------+-------+----------
 public | candidate_movies      | table | postgres
 public | comments              | table | postgres
 public | favorites             | table | postgres
 public | movie_embeddings      | table | postgres
 public | ratings               | table | postgres
 public | trailer_events        | table | postgres
 public | user_events           | table | postgres
 public | user_profiles         | table | postgres
 public | user_recommendations  | table | postgres
 public | users                 | table | postgres
(10 rows)
```

8) Öneri Sistemi Nasıl Çalışıyor?

Öneri sistemi, **“içerik tabanlı (content-based) + kullanıcı davranış sinyali ağırlıklı (behavior-driven)”** hibrit bir yaklaşımla çalışır. Buradaki temel fikir şudur:

- Filmlerin içerikleri (başlık, açıklama vb.) üzerinden **embedding vektörü** elde edilir.
- Kullanıcının yaptığı etkileşimlerden (favori/puan/trailer) bir **kullanıcı profili vektörü (user_vec)** oluşturulur.
- Kullanıcı vektörü ile aday film vektörleri arasında **benzerlik (similarity)** hesaplanır ve en iyi eşleşen filmler önerilir.

Bu yapı sayesinde sistem:

- Soğuk başlangıç (yeni kullanıcı) durumunda bile “popular/trending” gibi havuzlar üzerinden öneri üretebilir,
- Kullanıcı sinyali arttıkça kişiselleştirme kalitesi yükselir,
- Hesaplama maliyeti cache ile kontrol altında tutulur.

8.1 Kullanıcı Sinyalleri

Sistem, kullanıcıdan 3 ana davranış sinyali toplar:

1) Favorites (Favoriler)

- En güçlü pozitif sinyaldir.
- Kullanıcı “bunu seviyorum” mesajını açık şekilde verir.
- Öneri profilinde ağırlığı yüksek tutulur.

```
cur.execute(
    "SELECT movie_id, created_at FROM favorites WHERE user_id=%s ORDER BY id DESC LIMIT 60",
    (uid,),
)
for r in cur.fetchall():
    mid = r["movie_id"]
    decay = calculate_decay(r["created_at"])
    weights[mid] = weights.get(mid, 0.0) + (2.5 * decay)
```

2) Ratings (Puanlama / Like-Dislike)

- Pozitif ve negatif sinyal taşır.
- Örn: value = 1 → pozitif, value = -1 → negatif gibi.
- Sistem sadece “sevdiklerini” değil, “sevmediklerini” de öğrenir; bu da öneri kalitesini artırır.

```
cur.execute(
    "SELECT movie_id, value, created_at FROM ratings WHERE user_id=%s ORDER BY id DESC LIMIT 250",
    (uid,),
)
for r in cur.fetchall():
    mid = r["movie_id"]
    decay = calculate_decay(r["created_at"])
    base_val = 2.0 if r["value"] == 1 else -2.0
    weights[mid] = weights.get(mid, 0.0) + (base_val * decay)
```

3) Trailer Events (Trailer İzleme Etkinliği)

- “İlgi sinyali” olarak yorumlanır (implicit feedback).
- Aynı filmin trailer’ını tekrar tekrar izlemek, filme artan ilgi göstergesidir.
- İzleme sayısı arttıkça ağırlık artar ama bir noktadan sonra sınırlanır (aşırı baskın olmasın diye).

8.1.1 Ağırlıklandırma Mantığı

Her sinyal türü için sistem bir **ağırlık (weight)** hesaplar:

- Favorites → genellikle **daha yüksek pozitif ağırlık**
- Ratings → **pozitif veya negatif ağırlık**
- Trailer event → **log ölçekli veya sınırlı pozitif ağırlık**

Bu sayede tek bir sinyal türü tüm öneri sistemini domine etmez; dengeli bir profil çıkar.

8.1.2 Zaman Eskimesi (Decay)

Sistemde sinyallerin etkisi **zamanla azalır**:

- Yeni etkileşimler daha güçlü
- Eski etkileşimler daha zayıf

Bu şunu sağlar: Kullanıcının film zevki zamanla değiştiğinde, sistem bunu yakalayabilir.

Örnek mantık:

- Bugün favoriye eklenen film → yüksek etki
- 120 gün önce favoriye eklenen film → daha düşük etki

```
def calculate_decay(event_date):  HAKAN
    if not event_date:
        return 0.5
    delta = (now - event_date).days
    if delta < 0:
        delta = 0
    return 0.985 ** delta
```

8.2 Kullanıcı Profil Vektörü (User Profile)

Sistem, kullanıcı sinyallerini tek tek filmlere dağıttıktan sonra, kullanıcıyı temsil eden tek bir vektör üretir: **user_vec**.

Adım adım süreç:

1) Kullanıcının etkileşimde bulunduğu filmler bulunur

- favorites tablosu
- ratings tablosu

- trailer_events tablosu

2) Bu filmlerin embedding'leri çekilir

- Her film bir vektör (örneğin 384 boyutlu) ile temsil edilir.

3) Ağırlıklı ortalama ile kullanıcı vektörü çıkarılır

- Her film vektörü kendi sinyal ağırlığı ile çarpılır
- Toplanır, normalize edilir

4) Normalizasyon yapılır

- user_vec'in normu 1'e yakın olacak şekilde ölçeklenir
- Böylece benzerlik hesabı daha stabil olur

5) user_profiles tablosuna kaydedilir

- user_id, signals_hash, embedding, updated_at alanları ile

Bu yaklaşımın avantajı:e

- Her istek geldiğinde tüm sinyalleri tekrar hesaplamak yerine, **gerekli olduğunda profil üretilip saklanır.**

```
film_app=# SELECT
film_app-#   user_id,
film_app-#   signals_hash,
film_app-#   LEFT(embedding::text, 200) || ' ...' AS embedding_preview,
film_app-#   updated_at
film_app-# FROM user_profiles
film_app-# ORDER BY updated_at DESC
film_app-# LIMIT 3;
 user_id |               signals_hash               |                                                
```

Bu sayede:

- Hem güncel/popüler içerikler havuzda olur
- Hem hesaplama maliyeti sınırlanır (binlerce film yerine yüzlerce film)

8.3.1 Embedding Matrisi

Candidate pool içindeki filmlerin embedding'leri bir matriste tutulur:

- mat: (N x D) boyutlu film matrisi
    - N: candidate film sayısı
    - D: embedding boyutu

8.3.2 Benzerlik (Similarity) Skoru

Sistem, kullanıcı vektörü ile film matrisi arasında benzerlik hesaplar:

scores = mat @ user_vec

Bu yöntem pratikte şunu yapar:

- Her film için kullanıcıya "ne kadar yakın" olduğunu ölçer
- Skoru en yüksek olanlar top-N olarak seçilir

8.3.3 Görülen Filmlerin Filtrelenmesi

Sistem, kullanıcının zaten etkileşimde bulunduğu filmleri önermez:

- Favorilerde olanlar
- Puanlananlar
- Trailer izlenenler

Bu filmler bir "seen set" içinde toplanır ve öneri listesinden çıkarılır.

8.4 "Cache'den mi geldi yoksa taze mi?"

Bu proje öneri sisteminde **cache mantığı iki seviyede** uygulanır:

8.4.1 Kullanıcı Sinyal Hash (signals_hash)

Sistem kullanıcı sinyalinin değişip değişmediğini anlamak için bir "imza" üretir:

- favorites sayısı + son tarihi
- ratings sayısı + son tarihi
- trailer_events sayısı + son tarihi

Bunlar birleştirilir ve hashlenir (ör: SHA1). Bu hash değişmiyorsa demek ki kullanıcı sinyali değişmemiştir.

8.4.2 user_recommendations Cache

/api/personalized endpoint'i şu mantıkla çalışır:

A) Eğer sinyal değişmediyse:

- user_recommendations tablosundan hızlı şekilde döner
- "from_cache" gibi log düşer
- Performans çok yükselir (recompute yok)

B) Eğer sinyal değiştiyse:

- user profile yeniden hesaplanır
- candidate pool ile similarity yeniden çıkarılır
- top-N öneriler tekrar DB'ye yazılır
- "fresh" gibi log düşer

Bu yapı şu iki avantajı sağlar:

- **Hız:** aynı kullanıcı aynı sinyalle tekrar gelirse öneri saniyeler içinde döner
- **Doğruluk:** kullanıcı yeni favori ekler / rating yaparsa öneriler güncellenir

8.5 Öneri Sisteminin Güçlü Yönleri

- **Kişiselleştirme var:** her kullanıcıya farklı sonuç
- **Pozitif + negatif öğrenme:** sadece "sevilen" değil "sevilmeyen" de modelleniyor
- **Implicit sinyal desteği:** trailer izleme gibi doğal davranışlardan öğreniyor
- **Ölçeklenebilir tasarım:** candidate pool + cache ile maliyet kontrol ediliyor
- **İzlenebilir:** loglarla "from_cache/fresh" gibi durumlar görülebiliyor

8.6 Sınırlılıklar ve Geliştirme Önerileri (Sunumda Bonus Puan)

- Candidate pool genişletilebilir (daha fazla tür/ülke/period)

- Redis aktif kullanıma alınarak cache merkezi hale getirilebilir
- Kullanıcı sinyalleri çeşitlendirilebilir (arama geçmişi, sayfada kalma süresi vb.)
- A/B test ile öneri kalitesi ölçülebilir (offline evaluation + kullanıcı geri bildirimi)

1. "Sinyal → Ağırlık → Decay → user_vec" akış diyagramı
2. "Candidate pool → embedding matrix → dot product → topN → cache" diyagramı

9) Cache ve Performans Yaklaşımı

Projede hem **in-memory** hem de (tasarımsal olarak) **Redis cache** katmanı düşünülmüştür:

- **@lru_cache:** Tür listeleri gibi küçük ve sık kullanılan statik veriler için kullanılır.
- **Bellek içi aday havuzu (_mem_cand):** get_candidate_cache() ile candidate_movies tablosundan çekilen embedding matrisi RAM'de tutulur. Böylece her istekte veritabanından komple set çekilmez, öneri üretimi hızlanır.
- **Redis (tasarım/opsiyonel):** TMDB sonuçlarının ve embeddinglerin kısa süreli cache'i için docker-compose.yml içinde redis servisi tanımlanabilir. İleride yüksek trafik altında merkezi cache olarak kullanılabilir.

9.1 Loglama ile Performans İzleme Entegrasyonu

Bu projede cache yaklaşımının gerçekten işe yarayıp yaramadığını gözlemlemek için **loglama mimarisi** de sisteme entegre edilmiştir.

- API endpointleri çağrıldığında:
  - endpoint adı,
  - kullanıcı id (varsa),
  - IP bilgisi,
  - sonuç adedi (returned count),
  - "from_cache / fresh" gibi notlar loglara yazılır.

Örneğin /api/personalized çağrısında loglar şunu gösterir:

- "**from_cache**" → öneri daha önce üretildi, DB/cache üzerinden hızlı döndü
- "**fresh**" → öneri yeniden hesaplandı, candidate havuzu ve embedding hesapları çalıştı

Bu sayede sadece performans optimizasyonu yapmakla kalınmaz; aynı zamanda bu optimizasyonun etkisi sistem loglarından ölçülebilir.

“Hem cache hem Redis mantıklı mı?”

Evet, çünkü **amaçları farklı:**

- In-memory cache: tek container içinde ultra hızlı
- Redis: birden fazla web instance olursa ortak cache (dağıtık)

Bu projede **aktif olarak in-memory cache var**, Redis ise **opsiyonel/geleceğe dönük** tasarım bileşeni olarak düşünülür.

10) Loglama ve İzlenebilirlik

Projede loglama, hem hata ayıklama hem de güvenlik/izleme amacıyla kullanılır.

10.1 Neler loglanıyor?

- API çağrıları: endpoint + ip + sonuç
- Auth işlemleri: login success/fail, register fail (duplicate)
- Öneri endpoint’i: from_cache / fresh ayrımı
- Önemli kullanıcı eventleri: trailer izleme, logout vb.

10.2 Logların sağladığı fayda

- Hataları hızlı tespit
- Kullanıcı davranışı analizi (hangi akışlar kullanılıyor?)
- Performans analizi (cache çalışıyor mu?)
- Güvenlik analizi (başarısız login denemeleri)

```
hakan@192 bitirmeprojesi % docker compose logs -f web
web-1  | [2025-12-11 11:23:07 +0300] [1] [INFO] Starting gunicorn 22.0.0
web-1  | [2025-12-11 11:23:07 +0300] [1] [INFO] Listening at: http://0.0.0.0:5000 (1)
web-1  | [2025-12-11 11:23:07 +0300] [1] [INFO] Using worker: gthread
web-1  | [2025-12-11 11:23:07 +0300] [7] [INFO] Booting worker with pid: 7
web-1  | /usr/local/lib/python3.11/site-packages/transformers/utils/hub.py:127: FutureWarning: Using `TRANSFORMERS_CACHE` is deprecated and will be removed in v5 of
s. Use `HF_HOME` instead.
web-1  |   warnings.warn(
web-1  | [2025-12-11 11:28:26] [INFO] app.blueprints.api - API /api/featured called ip=146.75.118.132
web-1  | [2025-12-11 11:28:26] [INFO] app.blueprints.api - API /api/featured success returned=20
web-1  | [2025-12-11 11:28:26] [INFO] app.blueprints.api - API /api/featured called ip=146.75.118.132
web-1  | [2025-12-11 11:28:27] [INFO] app.blueprints.api - API /api/featured success returned=20
web-1  | [2025-12-11 11:29:21] [WARNING] app.blueprints.auth - Login FAILED email=hakankocaeli15@gmail.com ip=146.75.118.132
web-1  | [2025-12-11 11:29:41] [WARNING] app.blueprints.auth - Register FAILED (duplicate): username=hakan15 email=hakankocaeli15@gmail.com ip=146.75.118.132
```

11) Güvenlik Yaklaşımı

Projede kullanıcı hesaplarını ve kritik API işlemlerini korumak için temel güvenlik önlemleri uygulanmıştır. Bu yaklaşım; **şifre güvenliği**, **oturum yönetimi**, **yetkilendirme** ve **şüpheli giriş denemelerinin izlenmesi** üzerine kuruludur.

- **Şifrelerin güvenli saklanması:** Kullanıcı şifreleri veritabanında kesinlikle düz metin tutulmaz. Kayıt (register) sırasında şifre generate_password_hash() ile hash'lenerek password_hash alanına kaydedilir. Giriş (login) sırasında ise check_password_hash() ile girilen şifre doğrulanır. Böylece veritabanı ele geçirilse bile şifrelerin doğrudan okunması engellenir.
- **Session bazlı kimlik doğrulama:** Kullanıcı giriş yaptığında session["user_id"] atanarak oturum başlatılır. Çıkış (logout) işleminde session temizlenir (session.pop). Bu sayede kullanıcı giriş durumu güvenli biçimde takip edilir ve sayfalar/işlemler buna göre kontrol edilir.
- **Korunan endpoint'ler (login gerekli alanlar):** Kullanıcıya özel veya kritik işlemler içeren endpoint'ler login_required mantığıyla korunur. Örneğin /api/personalized ve /api/trailer_event gibi endpoint'ler yalnızca giriş yapmış kullanıcılar tarafından erişilebilir. Giriş yapılmamışsa kullanıcı login sayfasına yönlendirilir veya erişim reddedilir.
- **Hatalı giriş denemelerinin loglanması:** Başarısız login denemeleri hem uygulama loglarına hem de event kayıt sistemine yazılır (login_fail). Loglarda IP bilgisi tutulur ve e-posta gibi bilgiler doğrudan yazılmak yerine hash'lenerek (örn. sha1(email)) kaydedilir. Bu, brute-force denemelerini temel seviyede izlemeye ve şüpheli aktiviteleri tespit etmeye yardımcı olur

```python
# ------------------------------
# INVALID LOGIN
# ------------------------------
if not u or not check_password_hash(u["password_hash"], password):
    log_event( event_type: "login_fail",  payload: {"email_hash": sha1(email)})
    logger.warning(
        msg: "Login FAILED email=%s ip=%s",
        *args: email, request.remote_addr
    )
    flash("Geçersiz e-posta veya şifre.", "error")
    return redirect(url_for("auth.login"))

# ------------------------------
# SUCCESS LOGIN
# ------------------------------
session["user_id"] = u["id"]
log_event( event_type: "login_success",  payload: {"user_id": u["id"]})

logger.info(
    msg: "Login SUCCESS user_id=%s email=%s ip=%s",
    *args: u["id"], email, request.remote_addr
)
```

“Şifreler hash ile doğrulanır, başarısız girişler event/log olarak kaydedilir ve başarılı girişte session tabanlı oturum başlatılır.”

12) REST API Kullanımı ve Endpoint’ler

Projede frontend (JavaScript tabanlı arayüz) ile backend (Flask) arasındaki iletişim, **REST mimarisi** prensiplerine uygun olarak tasarlanmış **JSON tabanlı API endpoint’leri** üzerinden sağlanmaktadır. Frontend tarafı; sayfa yenilemeden veri çekmek (listeleme, arama önerisi, keşif filtreleri) ve kullanıcı aksiyonlarını kaydetmek (trailer izleme gibi) için bu endpoint’leri fetch / AJAX çağrıları ile kullanmaktadır.

API tasarımında:

- **GET** metodları veri okuma (listeleme, arama, keşif),
- **POST** metodları ise veri yazma ve kullanıcı event üretimi amacıyla kullanılmıştır.

Ayrıca endpoint’lerde **temel loglama** uygulanmıştır. Her istekte:

- istemci IP bilgisi,
- gelen parametreler,
- dönen sonuç sayıları loglanarak sistemin izlenebilirliği artırılmıştır.

Kullanıcıya özel işlemler içeren endpoint’ler login_required mekanizması ile korunmuş, böylece yalnızca oturum açmış kullanıcıların bu işlemleri gerçekleştirmesi sağlanmıştır.

12.1 Örnek Endpoint'ler ve Amaçları

GET /api/featured

Ana sayfada gösterilecek vitrin içeriklerini döndürür. Bu endpoint, TMDB servisinden **popular** ve **now_playing** kaynaklarını birleştirerek veri toplar. Görseli olmayan filmler elenir ve tekrar eden kayıtlar filtrelenerek yaklaşık **20 adet film** frontend'e iletilir.

```
bp.get("/api/featured")  HAKAN *
def api_featured():
    logger.info( msg: "API /api/featured called ip=%s",  *args: request.remote_addr)

    popular = tmdb_get( path: "/movie/popular",  params: {"page": 1})
    nowp    = tmdb_get( path: "/movie/now_playing",  params: {"page": 1})

    seen, results = set(), []
    for lst in (popular.get("results", []) + nowp.get("results", [])):
        mid = lst.get("id")
        if mid and mid not in seen and (lst.get("backdrop_path") or lst.get("poster_path")):
            seen.add(mid)
            results.append(lst)
        if len(results) >= 20:
            break

    logger.info( msg: "API /api/featured success returned=%s",  *args: len(results))
    return jsonify({"results": results})
```

GET /api/search_suggest?q=

Header alanında kullanılan **canlı arama (autocomplete)** özelliği için geliştirilmiştir. Kullanıcının girdiği q parametresi ile TMDB araması yapılır ve hızlı yanıt süresi sağlamak amacıyla sonuçlar **ilk 8 film** ile sınırlandırılır.

Dönen veri yalnızca özet alanları içerir:

- film id,
- başlık,
- poster,
- puan,
- yayın tarihi.

Amaç: Kullanıcıya hızlı ve akıcı bir arama deneyimi sunmak.

```python
@bp.get("/api/search_suggest")  HAKAN *
def api_search_suggest():
    q = (request.args.get("q") or "").strip()
    logger.debug( msg: "API /api/search_suggest q='%s' ip=%s",  *args: q, request.remote_addr)

    if not q:
        return jsonify({"results": []})

    data = tmdb_get( path: "/search/movie",  params: {"query": q, "page": 1, "include_adult": False})

    results = []
    for m in (data.get("results") or [])[:8]:
        results.append({
            "id": m.get("id"),
            "title": m.get("title"),
            "poster_path": m.get("poster_path"),
            "vote_average": m.get("vote_average"),
            "release_date": m.get("release_date"),
        })

    logger.debug( msg: "API /api/search_suggest returned %d items",  *args: len(results))
    return jsonify({"results": results})
```

GET /api/discover?genre_id=&year=&sort_by=&vote_gte=&page=

Keşif (discover) sayfasında filtreli listeleme sağlar. Tür, yıl, sıralama kriteri ve minimum puan gibi parametreler kullanılarak TMDB'nin discover endpoint'i çağrılır.

Bu yapı sayesinde frontend tarafında:

- dinamik filtreleme,
- sayfalama,
- sıralama işlemleri gerçekleştirilebilir.

Amaç: Kullanıcının tercihine göre özelleştirilebilir film keşfi sağlamak.

POST /api/trailer_event *(login gerekli)*

Kullanıcı bir filmin trailer'ını izlediğinde bu endpoint çağrılır. İlgili izleme bilgisi **trailer_events** tablosuna kaydedilir ve bu aksiyon bir **kullanıcı davranış sinyali (implicit feedback)** olarak değerlendirilir.

Ayrıca:

- event loglanır,
- kullanıcının öneri cache'i geçersiz kılınır (invalidate_user_cache).

Amaç: Kullanıcı davranışlarını event olarak toplayarak öneri sistemine sinyal üretmek.

GET /api/personalized *(login gerekli)*

Kullanıcıya özel kişiselleştirilmiş film önerilerini döndürür.

Çalışma mantığı:

1. Kullanıcının signals_hash değeri ile **user_recommendations** tablosunda cache kontrol edilir.
2. Cache varsa sonuçlar **from_cache** olarak hızlı şekilde döndürülür.
3. Cache yoksa:
    a. aday film havuzu üzerinden benzerlik hesaplanır,
    b. top-N film seçilir,
    c. sonuçlar veritabanına kaydedilir ve **fresh** olarak döndürülür.
4. Kullanıcının daha önce etkileşimde bulunduğu filmler (favori, puan, trailer) öneri listesinden elenir.

Amaç: Performanslı ve güncel kişiselleştirilmiş öneriler sunmak.

12.2 Bu Yaklaşımın Tercih Edilme Nedenleri

- Frontend tarafında **hızlı ve esnek veri erişimi** (AJAX / fetch).
- Kullanıcı aksiyonlarını **event tabanlı** toplayarak öneri sistemine sinyal üretme.
- **Cache mekanizması** sayesinde performans ve ölçeklenebilirlik.
- login_required ile kullanıcıya özel endpoint'lerde **güvenlik** sağlanması.
- Loglama sayesinde hata ayıklama ve sistem izleme kolaylığı.

13) Docker ile Kurulum ve Çalıştırma

Projeyi çalıştırma adımları:

1. .env oluştur (TMDB key vb.)
2. docker compose up --build
3. Web: http://localhost:5002

Docker Compose servisleri:

- db: Postgres (5433 dış port)

- web: Flask + Gunicorn (5002 dış port)
- (opsiyonel) redis: cache için

```
akan@192 bitirmeprojesi % docker compose ps
AME                  IMAGE                COMMAND                  SERVICE   CREATED      STATUS                    PORTS
itirmeprojesi-db-1    postgres:16-alpine   "docker-entrypoint.s…"   db        3 weeks ago  Up 32 minutes (healthy)   0.0.0.0:5433->5432/tcp, [::]:5433->5432/tcp
itirmeprojesi-web-1   bitirmeprojesi-web   "/entrypoint.sh guni…"   web       3 weeks ago  Up 32 minutes             0.0.0.0:5002->5000/tcp, [::]:5002->5000/tcp
```

14) Test Süreci (Selenium + Pytest)

Bu projede, kullanıcı arayüzünün (UI) temel akışlarının gerçekten çalıştığını doğrulamak için **Selenium tabanlı uçtan uca (E2E)** testler eklenmiştir. E2E test yaklaşımında amaç; yalnızca bir fonksiyonun doğru çalıştığını değil, kullanıcının tarayıcı üzerinden yaptığı işlemlerin (sayfa açılması, form gönderimi, yönlendirme, giriş/çıkış vb.) **gerçek tarayıcı davranışıyla** doğrulanmasıdır.

Bu sayede:

- UI tarafındaki kırılmalar (buton görünmüyor, selector değişti, form action bozuldu vs.)
- Backend endpoint'lerinin erişilebilirliği
- Sayfalar arası yönlendirme problemleri erken aşamada yakalanır.

14.1 Testlerin Amacı

Testler iki ana hedefe odaklanır:

A) UI doğrulaması (tarayıcı üzerinden)

- **Ana sayfa açılıyor mu?** (title kontrolü + sayfanın yüklenmesi)
- **Header/menü elemanları var mı?** (logo, arama input'u gibi kritik bileşenler)
- **Arama formu doğru çalışıyor mu?** (kullanıcı metin yazıp Enter basınca /search sayfasına gitmesi)
- **/search sayfası içerik üretiyor mu?** (body içinde sonuç/film/aranan kelime gibi içerik varlığı)
- **Canlı arama önerisi paneli açılıyor mu?** (searchSuggest panelinin hidden olmaktan çıkması)
- **Login/Register sayfaları render oluyor mu?** (form alanları görünür mü?)

- **Favorites sayfası login gerektiriyor mu?** (login'e yönlendirme veya "giriş" uyarısı)

B) API sağlığı (requests ile hızlı kontrol)

- /api/featured endpoint'i 200 dönüyor mu ve results listesi var mı?
- /api/search_suggest?q=... 200 dönüyor mu ve results listesi var mı?

Not: UI testleri "kullanıcı gibi" tarayıcıyla çalışırken, API testleri daha hızlı bir şekilde backend'in ayakta olduğunu doğrulamak için eklenmiştir.

14.2 Çalıştırma Mantığı

Bu testler, geliştirme ortamında iki parçanın birlikte çalışmasına dayanır:

1. **Uygulama (Flask + Frontend) Docker üzerinde çalışır** Varsayılan hedef adres: http://localhost:5002
2. **Testler lokal Python ortamında (.venv) çalıştırılır** Bu sayede test ortamı izole olur ve CI/CD süreçlerine de kolay taşınabilir.

14.2.1 Hedef Ortam Seçimi (APP_BASE_URL)

Testlerin hangi adrese gideceği APP_BASE_URL ile belirlenir.

- Docker ortamı: export APP_BASE_URL=http://localhost:5002
- Lokal çalıştırma (opsiyonel): export APP_BASE_URL=http://localhost:5000

Ardından testleri çalıştırma:

- pytest -q

14.3 Test Altyapısı (conftest.py) – Nasıl Çalışıyor?

tests/conftest.py dosyası, tüm testlerde ortak kullanılan altyapıyı kurar.

a) Driver kurulumu (Chrome)

- Selenium testleri gerçek bir Chrome tarayıcı başlatır.
- webdriver_manager ile ChromeDriver otomatik indirilir/ayarlanır (manuel kurulum ihtiyacını azaltır).
- Varsayılan olarak **headless** çalışır (ekran açmadan test koşar).

b) Headless / görünür mod

- Varsayılan: HEADLESS=1 → headless
- Eğer tarayıcıyı ekranda görmek istersen:
    - export HEADLESS=0

c) Bekleme (Wait) mantığı UI testlerinde en kritik konu: "element daha yüklenmeden tıklama/yazma denemek" hatasıdır. Bu yüzden WebDriverWait + expected_conditions kullanılır.

- Varsayılan bekleme süresi: 15 saniye
- İstersen değiştirebilirsin:
    - export SELENIUM_WAIT=20

d) Hata anında otomatik ekran görüntüsü Bir test fail olursa:

- test_artifacts/screenshots/ içine otomatik screenshot kaydedilir.
- Dosya adında test adı + timestamp bulunur.

Bu özellik, özellikle CI ortamında "neden fail oldu?" sorusunu hızlı çözmek için kritik.

14.4 Test Senaryoları – Neleri Doğruluyoruz?

Projede bulunan örnek testler şunları hedefler:

- **test_homepage_loads:** Ana sayfa açılır, title beklenen ifadeyi içerir.
- **test_header_elements_exist:** Logo ve arama input'u sayfada mevcut mu?
- **test_nav_listeler_link_works:** "Listeler" linki /search sayfasına götürüyor mu?
- **test_search_form_redirects_to_search_page:** Arama input'una yazıp Enter basınca /search açılıyor mu?
- **test_search_results_page_has_content:** /search?q=matrix açılınca sayfa boş değil mi?
- **test_live_search_suggest_panel_opens:** canlı arama paneli görünür hale geliyor mu?
- **test_login_page_loads / test_register_page_loads:** login/register form alanları var mı?
- **test_api_featured_health / test_api_search_suggest_health:** backend endpoint'leri sağlıklı mı?
- **test_favorites_page_requires_login_or_redirects:** favoriler sayfası login gerektiriyor mu?

- **test_login_and_logout_flow_if_credentials_provided:** eğer env ile test hesabı verilirse login+logout uçtan uca çalışır

14.5 Opsiyonel: Login Testi için Test Kullanıcısı

Login/logout testi, gerçek kullanıcı bilgisi gerektirir. Bu yüzden credentials verilmezse test “pas geçer”.

Kullanmak istersen:

- export TEST_USER_EMAIL="..."
- export TEST_USER_PASSWORD="..."

Bu sayede test, login olup “Çıkış” linkini görmeyi ve ardından logout sonrası “Giriş” linkini doğrular.

14.6 Neden Bu Yapı Tercih Edildi?

- **Gerçek tarayıcı ile doğrulama:** UI kırılmaları hızlı yakalanır.
- **Wait + EC kullanımı:** flakiness (rastgele fail) azaltılır.
- **Headless default:** hızlı koşum + CI uyumu.
- **Screenshot on fail:** hata analizi kolaylaşır.
- **APP_BASE_URL ile esneklik:** aynı testler hem Docker hem lokal ortamda çalışır.

```
(.venv) hakan@192 bitirmeProjesi % pytest -q
............
12 passed in 13.33s
```

15) CI/CD Durumu ve Önerilen Akış

Şu an aktif bir CI/CD pipeline zorunlu olarak kurulmuş değil (proje yerel + docker ile çalışıyor). Ancak önerilen CI/CD:

- GitHub Actions
- push → unit+ui test → docker build → (opsiyonel) deploy

16) Projenin Güçlü Yanları

- Kişiselleştirme mantığı gerçek sinyallere dayanıyor
- Cache yaklaşımıyla performans düşünülmüş
- Loglama ile izlenebilirlik güçlü
- Docker ile taşınabilir ve hızlı kurulum
- REST endpointleri sayesinde UI daha dinamik
- Selenium E2E testleriyle temel kullanıcı akışları doğrulanıyor

17) Limitler ve İyileştirme Fikirleri

- “Candidate pool” TMDB kaynaklı; tam katalog yok
- Redis şu an opsiyonel (yük altında devreye alınabilir)
- Test kapsamı daha da büyütülebilir (login/register/favorite/personalized)
- CI/CD eklenerek otomatik kalite kapısı kurulabilir
- Rate limit / caching stratejisi daha rafine hale getirilebilir

18) Sonuç

Bu proje; modern web geliştirme, veri tabanı tasarımı, REST API, öneri sistemleri, cache ve loglama gibi kritik yazılım mühendisliği konularını tek bir uygulama üzerinde birleştirmiştir. Tasarım “geliştirilebilir ve ölçeklenebilir” şekilde kurgulanmış, ileride Redis/CI/CD gibi bileşenler ile daha üretim seviyesi bir sisteme evrilebilecek altyapı hazırlanmıştır.